

Java

File handling, array list, linkedlist

```
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class Filehandling {

    public static void main(String[] args) throws IOException {

        System.out.println("File handling");

        // volatile vs non-volatile memory
        // Ram vs hard drive

        // File file = new
File("E:\\Fiiit\\javaFullStack_sahana\\java\\src\\demo");

        // file.mkdir(); // folder

        File file1 = new
File("E:\\Fiiit\\javaFullStack_sahana\\java\\src\\demo\\file1\\file2");

        file1.mkdirs();

        File file2 = new
File("E:\\Fiiit\\javaFullStack_sahana\\java\\src\\demo\\demo.txt");

        file2.createNewFile();

        File file3 = new File("example.txt");

        file3.createNewFile();

        if(file3.exists()){
            System.out.println(file3.getName());
            System.out.println(file3.getAbsolutePath());
            System.out.println(file3.canWrite());
            System.out.println(file3.canRead());
            System.out.println(file3.length());
        }else{
            System.out.println("File does not exist");
        }
    }
}
```

```

        try {
            FileWriter writer = new FileWriter("example.txt", true );

            writer.write("\n Hello, this is sample file ");
            writer.close();

        } catch (Exception e) {
            // TODO: handle exception
            e.printStackTrace();
        }

        try {
            Scanner sc = new Scanner(file3);

            while(sc.hasNextLine()){
                String data = sc.nextLine();
                System.out.println(data);
            }

        } catch (Exception e) {
            // TODO: handle exception
            e.printStackTrace();
        }

        System.out.println("End");
    }
}

// interface CollectionArray{
//     void add();
//     int data =10;
// }

// class Arraylistss implements CollectionArray{

//     public void add(){
//         System.out.println("Add method added");
//     }
// }

import java.util.ArrayList;
import java.util.LinkedList;

public class Collections {
    public static void main(String[] args) {
        System.out.println("Collections");
        // arraylist -> allow duplicate elements, maintain insertion order
    }
}

```

```
// non-synchronized , manipulation is very slow , access element is fast (index)
```

```
ArrayList<Integer> list = new ArrayList<>();
```

```
list.add(2);  
list.add(1);  
list.add(6);  
list.add(10);  
list.add(3);  
list.add(2);
```

```
// System.out.println(list[0]); X  
System.out.println(list.get(0));
```

```
System.out.println(list.size());  
list.remove(2);  
list.sort(null);
```

```
list.set(2, 20);
```

```
System.out.println(list.contains(21));  
// list.clear();
```

```
System.out.println(list);
```

```
// System.out.println(list.get(0));  
// int -> integer object
```

```
// List interface -> class ( arraylist, linkedlist)
```

```
LinkedList<String> li = new LinkedList<>();
```

```
li.add("Apple");  
li.add("Banana");  
li.add("Carrot");
```

```
li.addFirst("First");  
li.addLast("Last");
```

```
System.out.println(li.peek());
```

```
System.out.println(li.remove());
```

```
        System.out.println(li);
    }
}
```

Theory Notes

1. File Handling

- **Classes:**
 - File → represents file/directory.
 - FileWriter → writes text to files.
 - Scanner → reads text from files.
- **Common methods:**
 - createNewFile() → creates a new file.
 - mkdir() / mkdirs() → create directories.
 - exists() → check if file exists.
 - delete() → delete file.
 - canRead(), canWrite() → permissions.
 - length() → file size.

2. Collections Framework

- **ArrayList** → dynamic array, allows duplicates, maintains insertion order, fast access by index.
- **LinkedList** → doubly linked list, efficient insert/delete, supports queue operations.
- **Common methods:**
 - add(), remove(), get(), set(), contains(), clear().
 - addFirst(), addLast(), peek(), remove() (specific to LinkedList).

Coding Round Questions

1. Write a program to create a new file and check if it exists. Print its name, path, and size.
2. Demonstrate writing text into a file using FileWriter.
3. Read file content line by line using Scanner.
4. Create an ArrayList<Integer> and perform add(), remove(), sort(), set(), and contains().
5. Create a LinkedList<String> and demonstrate addFirst(), addLast(), peek(), and remove().
6. Write a program to delete a file and check if it still exists.

Problem: Student Records Management System

Requirements:

- Store student records (name, marks) in a file.
- Use ArrayList to maintain student objects.
- Add methods to:
 - addStudent(name, marks) → adds record to list and file.
 - viewStudents() → reads from file and prints all records.
 - searchStudent(name) → checks if student exists in list.
- Demonstrate file persistence (data remains after program restarts).